

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



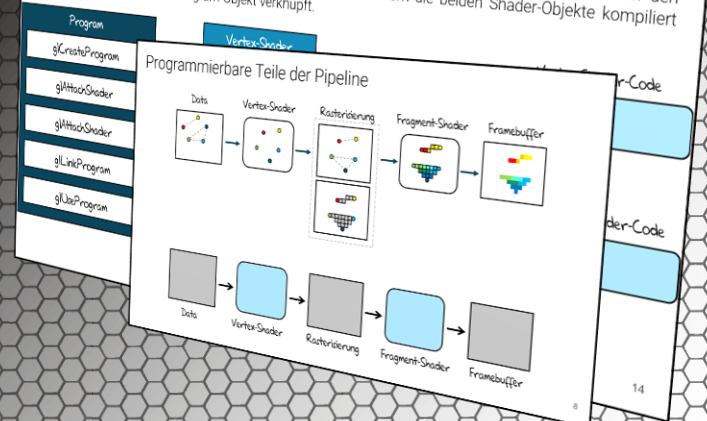
Einführung in Vertex- und Fragment-Shader

„Das Wasser nimmt jede Form an, in die man es gießt, und doch bleibt es sich selbst treu.“ (sinngemäß nach Katsushika Hokusai)

PD Prof. Dr. Marco Block-Berlitz

Shadercode in LWJGL einbinden – Übersicht

Die beiden Zeichenketten für die Vertex- und Fragment-Shader sind rechts zu sehen und werden in den jeweiligen Shader-Objekten mit `glShaderSource` gebunden. Nachdem die beiden Shader-Objekte kompiliert wurden, werden sie mit einem Program-Objekt verknüpft.



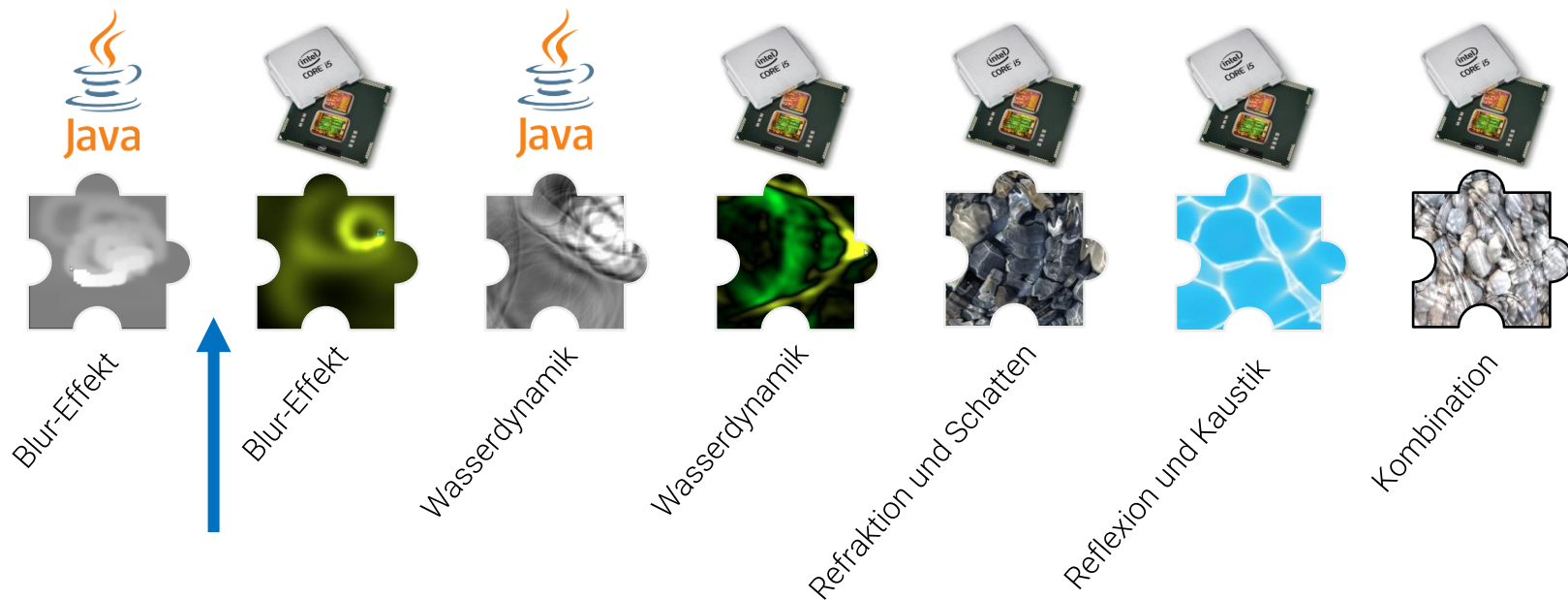
Ziele und Inhalt

- OpenGL als Zustandsmaschine
- Vereinfachte Renderpipeline in OpenGL
- Programmierbare Teile der Renderpipeline
- GLSL-Besonderheiten
- Operationen, semantische Bedeutung
- Konstruktoren
- Zusammenspiel von Matrizen und Vektoren
- Funktionen in GLSL
- Shadercode in LWJGL einbinden
- Beleuchtungsberechnung einer Szene
- Beispiel: Gelbes Quad rotieren lassen
- Beispiel: Quad mit Tiefe einfärben
- Beispiel: Quad mit konstanter Helligkeit
- Beispiel: Quad mit Licht- und Materialeigenschaften
- Beispiel: Torus-Eigenschaften live ändern
- Shader-Utilities
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



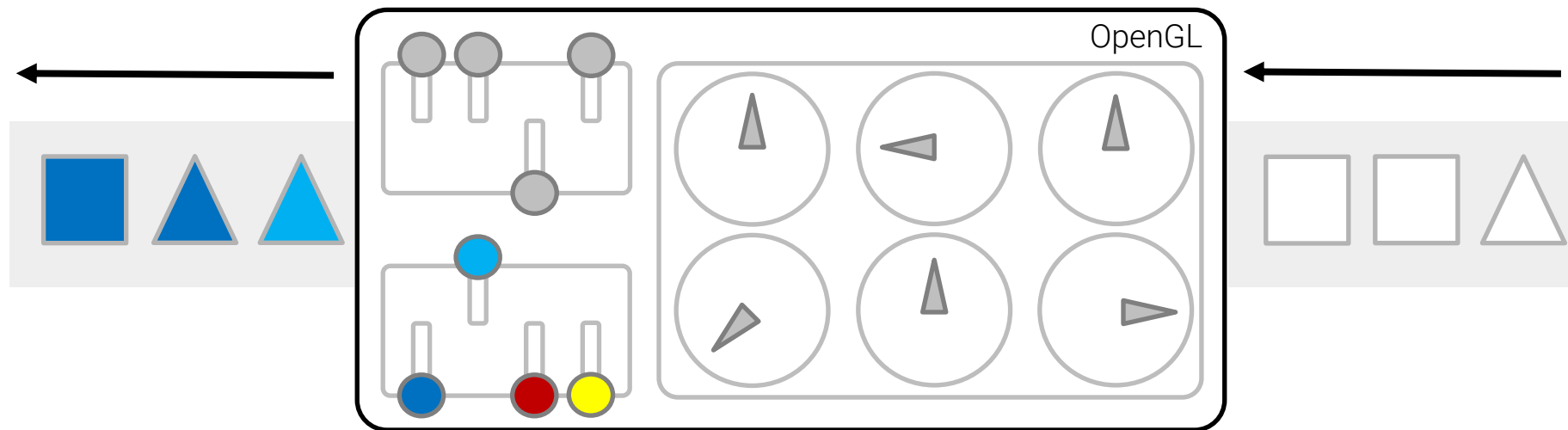
Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.



OpenGL als Zustandsautomat

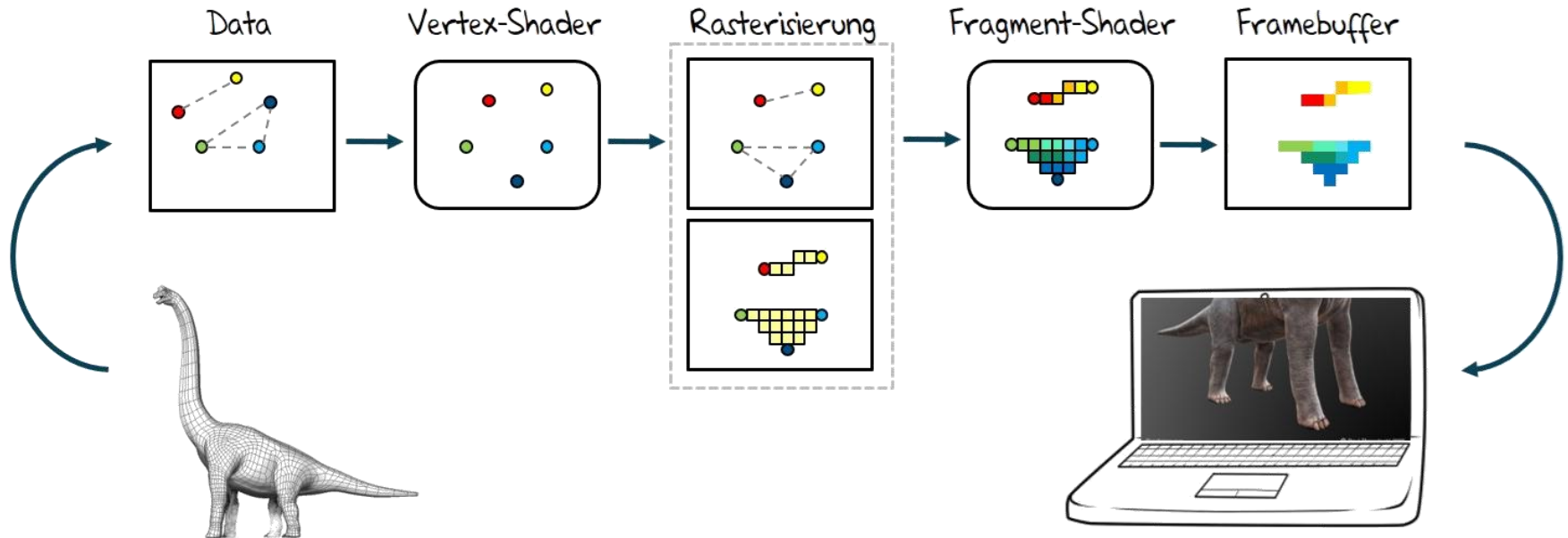
OpenGL wurde als [Zustandsautomat](#) realisiert. Es gibt eine Vielzahl von Parametern, die beschreiben, wie OpenGL mit vorhandenen Daten arbeiten soll. Das bedeutet, dass alle Parameter so lange Anwendung finden, bis sie verändert werden.



Stellen wir beispielsweise die aktuelle Farbe auf Hellblau, werden im Anschluss alle Vertices in Hellblau dargestellt. Durch dieses Konzept werden wir dazu gezwungen, uns mit den [Standardeinstellungen](#) des Zustandsautomaten zu beschäftigen. Um Programmpassagen austauschbar zu machen, sollten im Anschluss an die entsprechende Methode wieder die Standardeinstellungen verwendet werden.

Vereinfachte Renderpipeline - Zusammenfassung

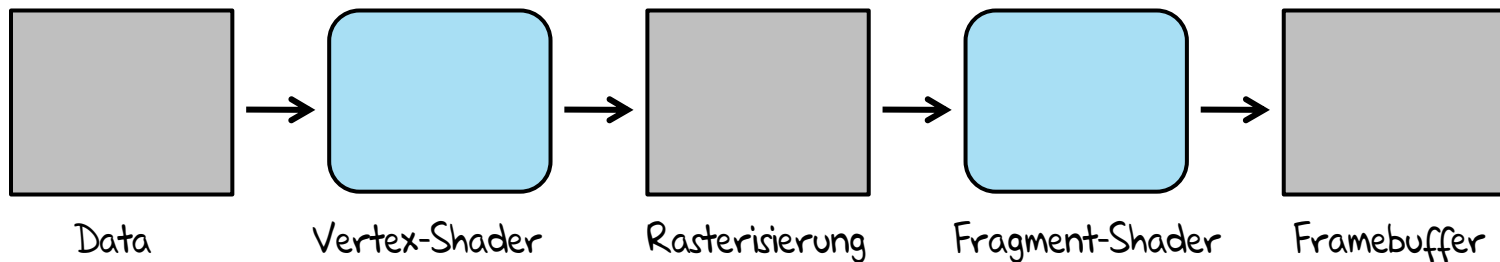
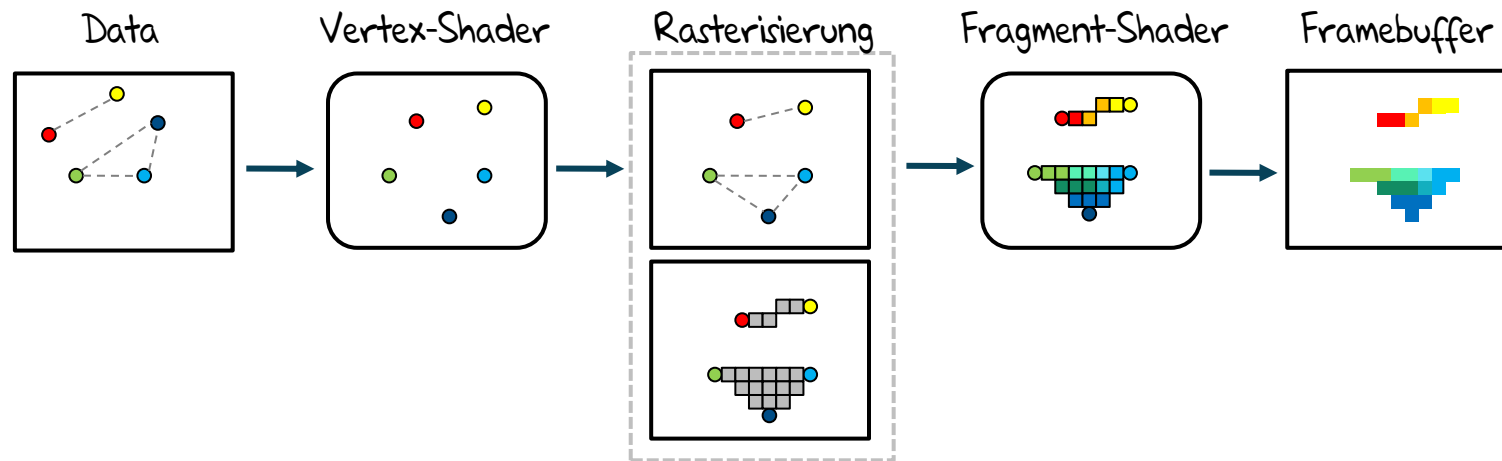
Typische vereinfachte Renderpipeline von OpenGL: Gegeben sind geometrische Primitive mit unterschiedlichen Informationen (links als Farben dargestellt). Anschließend werden diese durch den Vertex-Shader an die gewünschten, neuen Positionen transformiert. Nach dem Zielraster werden die Inhalte der Primitiven diskretisiert und die Inhalte linear interpoliert (mittig). Der Fragment-Shader übernimmt die Färbung der interpolierten Werte. Schließlich wählen wir den Ausgabebereich, z. B. den Bildschirm (rechts).





Schauen wir uns die Pipeline
noch einmal genauer an ...

Programmierbare Teile der Pipeline



Programmierbare Teile der Pipeline

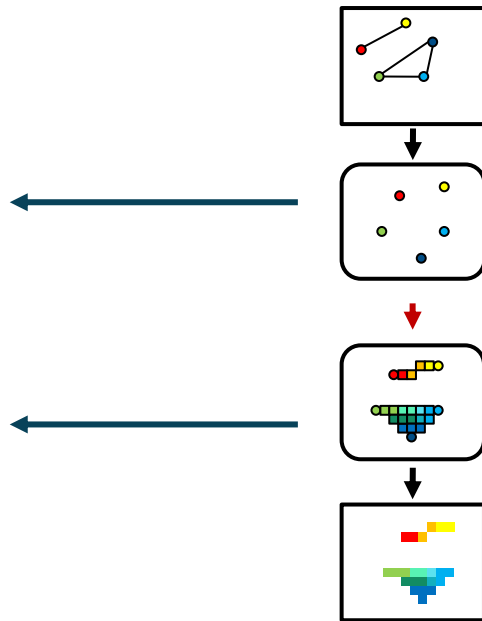
Schauen wir uns die beiden **programmierbaren Teile** (den Vertex- und den Fragment-Shader) der vereinfachten Pipeline als einfache Beispiele einmal an:

Vertex-Shader

```
void main(void) {  
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;  
}
```

Fragment-Shader

```
void main(void) {  
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Farbe Rot  
}
```



GLSL-Besonderheiten I

Operatoren können in GLSL intuitiv verwendet werden:

```
vec3 v1, v2;  
vec3 v3 = v1 + v2;
```

Vektoren haben nach dem ersten Zugriff auf die Komponenten eine semantische Bedeutung:

```
vec4 col, pos, tex;  
vec4 col2 = vec4(col.r, col.g, col.b, col.a); //Farbe  
vec4 pos2 = vec4(pos.x, pos.y, pos.z, pos.w); //Position  
vec4 tex2 = vec4(tex.s, tex.t, tex.p, tex.q); //Textur
```

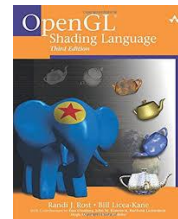
Der Compiler verwendet im Hintergrund nur einen Vektortyp:

```
vec2 redGreen = col.rg;  
vec3 pos_xyz = pos.xyz;
```

Multiples Auslesen ist möglich (Compiler achtet auf Semantik!):

```
vec4 v2 = v.xxyy;
```

```
vec4 v2 = v.xgba;
```



Rost R. J., et al.: „OpenGL Shading Language“, 3. Auflage, Addison-Wesley-Verlag, 2009 „orange book“

GLSL-Besonderheiten II

Typumwandlungen sind nur über **Konstruktoren** möglich, dabei wird eine ganze Reihe von Kombinationsmöglichkeiten angeboten:

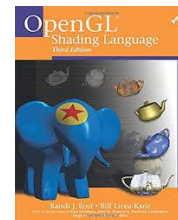
```
vec3 coord_xyz    = vec3(1.0, 0.0, -0.5);  
vec4 coord_xyzw   = vec4(coord_xyz, 1.0);
```

Hier noch ein paar abschließende Beispiele für die **Indizierung**:

```
vec4 v            = vec4(1.0, 2.0, 3.0, 4.0);  
float f          = v[2];           // liefert Element 3.0  
v.xw              = vec2(8.0, 9.0);
```

Nach dem letzten Beispiel trägt der Vektor *v* die Werte (8.0f, 2.0f, 3.0f, 9.0f). Der Umgang mit Matrizen und das **Zusammenspiel von Matrizen und Vektoren** sind ebenfalls sehr komfortabel und intuitiv in GLSL gelöst:

```
mat4 M            = mat4(3.0);    // Diagonale wird auf 3.0 gesetzt  
vec4 v            = M[1];         // zweite Spalte wird kopiert
```



Rost R. J., et al.: „OpenGL Shading Language“, 3. Auflage, Addison-Wesley-Verlag, 2009 „orange book“

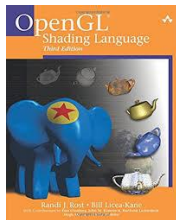
GLSL-Besonderheiten II

Analog zu fast allen gängigen imperativen Programmiersprachen ist es in GLSL ebenfalls möglich, Programmteile in **Funktionen** auszulagern. In dem folgenden Beispiel wollen wir zeigen, wie die Funktionen **TranslateMatrix** und **ScaleMatrix**, die jeweils aus einem Vektor die entsprechende homogene Matrix erzeugen, in einem Vertex-Shader erstellt werden können:

```
mat4 TranslateMatrix(const in vec3 t){
    return mat4(1.0, 0.0, 0.0, 0.0,
                0.0, 1.0, 0.0, 0.0,
                0.0, 0.0, 1.0, 0.0,
                t.x, t.y, t.z, 1.0);
}

mat4 ScaleMatrix(const in vec3 s){
    return mat4(s.x, 0.0, 0.0, 0.0,
                0.0, s.y, 0.0, 0.0,
                0.0, 0.0, s.z, 0.0,
                0.0, 0.0, 0.0, 1.0);
}

void main(void) {
    float sFac = 0.5;
    mat4 SCALE_M = ScaleMatrix(vec3(sFac, sFac, sFac));
    vec4 res = SCALE_M * vec4(1.0, -1.0, 2.0, 1.0);
    ...
}
```



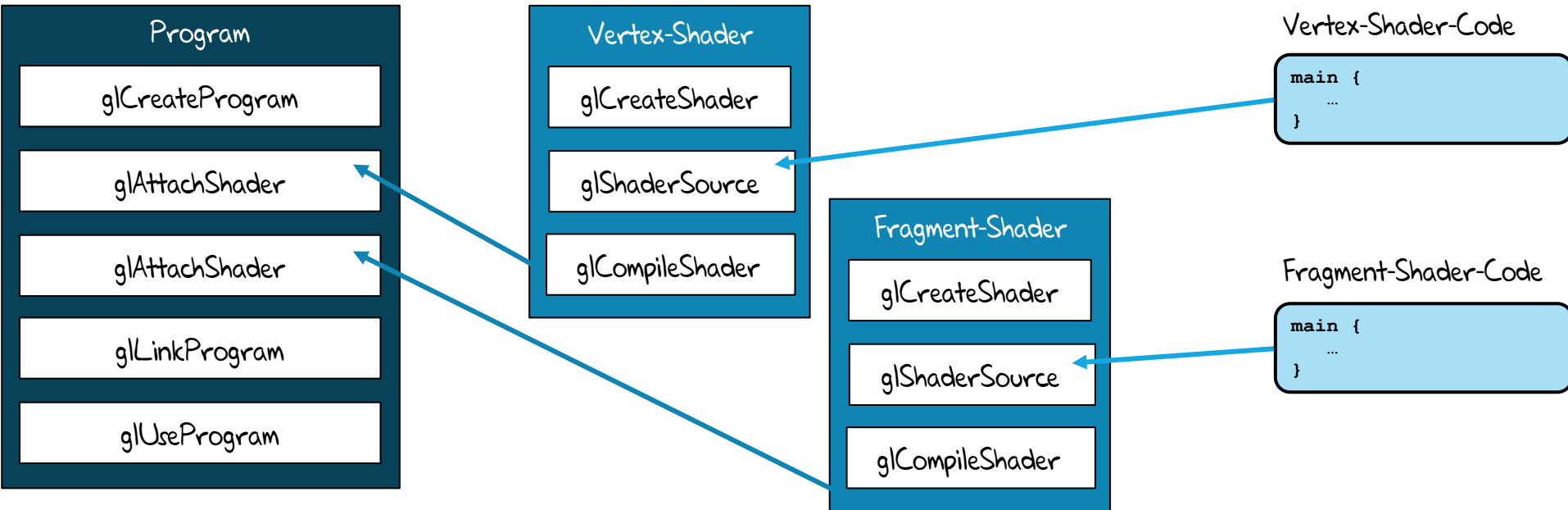
Rost R. J., et al.: „OpenGL Shading Language“, 3. Auflage, Addison-Wesley-Verlag, 2009 „orange book“



Shadercode mit LWJGL

Shadercode in LWJGL einbinden – Übersicht

Die beiden Zeichenketten für die Vertex- und Fragment-Shader sind rechts zu sehen und werden in den jeweiligen Shader-Objekten mit **glShaderSource** gebunden. Nachdem die beiden Shader-Objekte kompiliert wurden, werden sie mit einem Program-Objekt verknüpft.



Shadercode in LWJGL einbinden I

Dazu müssen wir den GLSL-Code für die Vertex- und Fragment-Shader zunächst in einem **String** ablegen:

```
String vertShader = ""
+ "void main(void) {"
+ "    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;"
+ "}";

String fragShader = ""
+ "void main(void) {"
+ "    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);"
+ "}";
```



Vertex-Shader-Code

```
main {
  ...
}
```

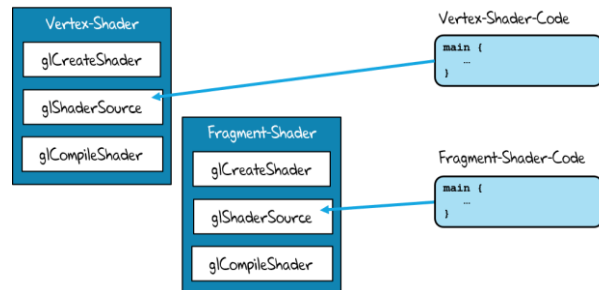
Fragment-Shader-Code

```
main {
  ...
}
```

Jetzt wollen wir die beiden **Shader-Objekte** erzeugen und die entsprechenden Zeichenketten binden und kompilieren:

```
int shaderObjectV = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(shaderObjectV, vertShader);
glCompileShader(shaderObjectV);

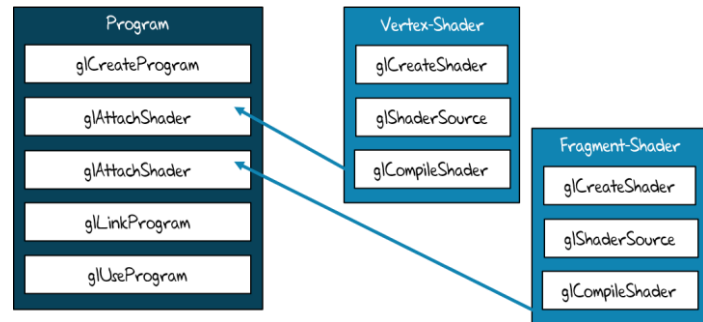
int shaderObjectF = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(shaderObjectF, fragShader);
glCompileShader(shaderObjectF);
```



Shadercode in LWJGL einbinden II

In **shaderObjectV** und **shaderObjectF** haben wir die Verlinkungen zu den beiden Shader-Objekten vorliegen und können diese in einem neuen **Programm-Objekt** zusammenführen. Dazu wird ein neues Program-Objekt **programObject** erzeugt und die Shader-Objekte über die Funktion **glAttachShader** verknüpft:

```
int programObject = glCreateProgram();  
glAttachShader(programObject, shaderObjectV);  
glAttachShader(programObject, shaderObjectF);  
glLinkProgram(programObject);
```



Mit **glUseProgram** können wir das erstellte Shader-Programm in die Grafikkarte laden und verwenden:

```
glUseProgram(programObject);
```

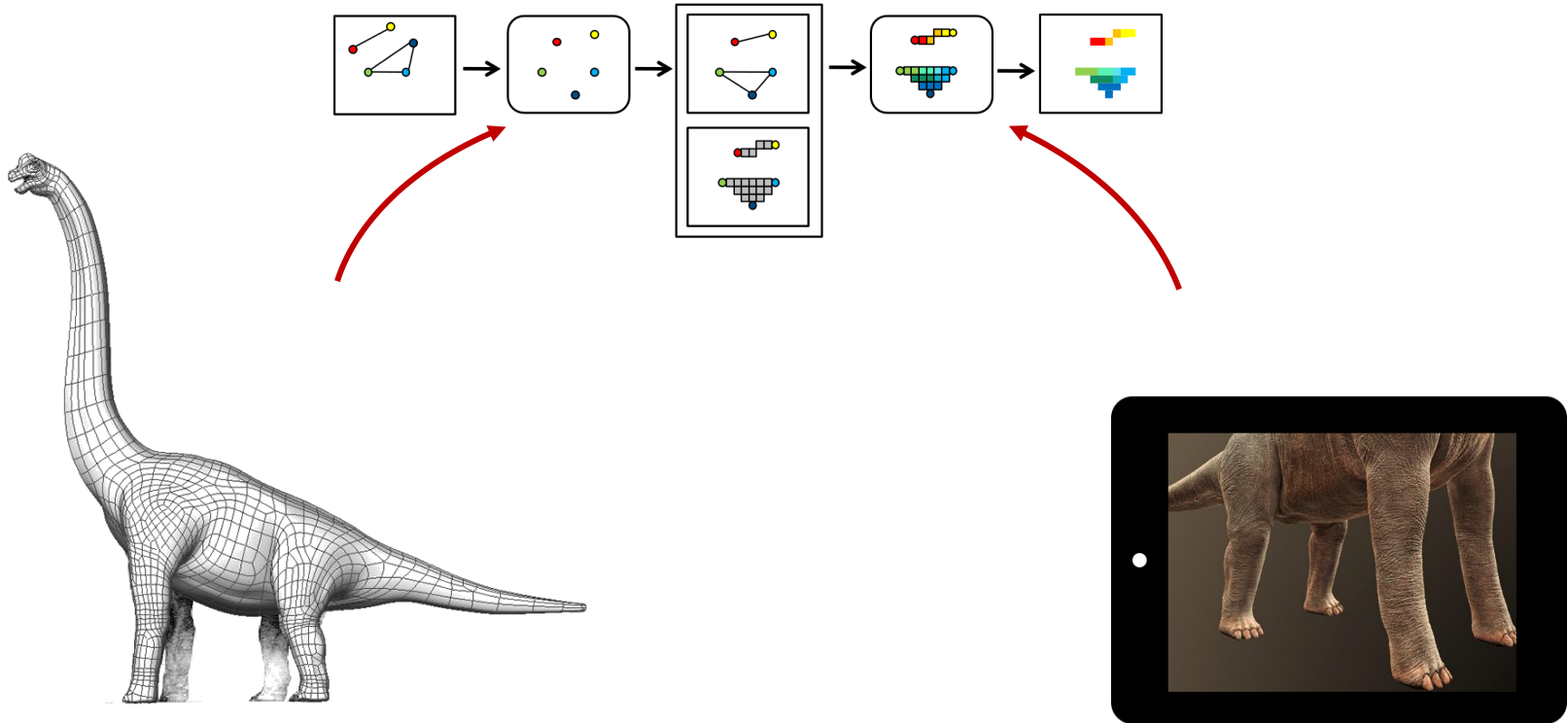
Alle folgenden OpenGL-Befehle durchlaufen dann die Pipeline mit unseren Varianten der Vertex- und Fragment-Shader.



Wo sollte die Berechnung der **Beleuchtung einer Szene** Eurer Meinung nach stattfinden?

Beleuchtungsberechnung einer Szene

Wo sollte die Berechnung der Beleuchtung einer Szene Eurer Meinung nach stattfinden?

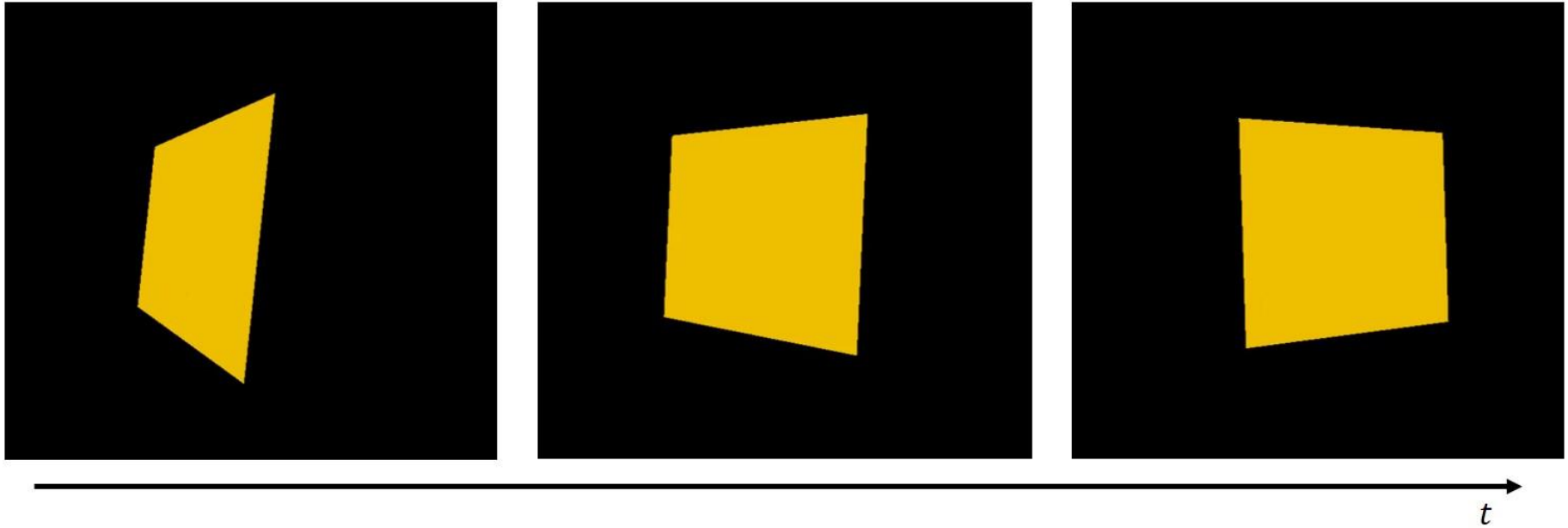




Beginnen wir die **Shaderprogrammierung** mit einem einfachen Beispiel.

Gelbes Quad rotieren lassen

Die Farbe des Quads soll an allen Stellen gleich sein, unabhängig davon, wie weit diese vom Betrachter entfernt sind. Das Beispiel wollen wir jetzt über einen Shader lösen.



Gelbes Quad rotieren lassen (Shader)

Es folgt ein einfaches Shader-Beispiel, bei dem ein Quad in der Mitte des Bildschirms rotiert und mit einer konstanten Farbe versehen wird. Beim [Vertex-Shader](#) reichen wir das Transformationsergebnis durch:

```
void main() {  
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;  
}
```

Der [Fragment-Shader](#) vergibt für jedes zu zeichnende Pixel eine Farbe. Im folgenden Beispiel ist es die Farbe Gelb, da Kanäle für Rot und Grün auf 1.0 und der entsprechende Kanal für Blau auf 0.0 gesetzt werden:

```
void main() {  
    gl_FragColor = vec4(1.0, 1.0, 0.0, 1.0);    // (r, g, b, a)  
}
```

Gelbes Quad rotieren lassen (Java)

Die beiden Shadercodepassagen bieten wir dabei als Zeichenketten an:

```
public class QuadGelb extends LWJGLBasisFenster {
    private static String vertexShaderSource = ""
        + "void main() {"
        + "    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;"
        + "}";

    private static String fragShaderSource = ""
        + "void main() {"
        + "    gl_FragColor = vec4(1.0, 1.0, 0.0, 1.0);"
        + "}";

    public QuadGelb() {
        super("Rotierendes Quad über Shader realisiert", 800, 600);
        initDisplay();
    }
}
```

Wir erzeugen das Program-Objekt in der Methode **prepareShader**:

```
private void prepareShader() {
    int myProgram = glCreateProgram();

    int vertShader = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertShader, vertexShaderSource);
    glCompileShader(vertShader);
    System.out.println(glGetShaderInfoLog(vertShader, 1024));
    glAttachShader(myProgram, vertShader);

    int fragShader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragShader, fragShaderSource);
    glCompileShader(fragShader);
    System.out.println(glGetShaderInfoLog(fragShader, 1024));
    glAttachShader(myProgram, fragShader);

    glLinkProgram(myProgram);
    glUseProgram(myProgram);

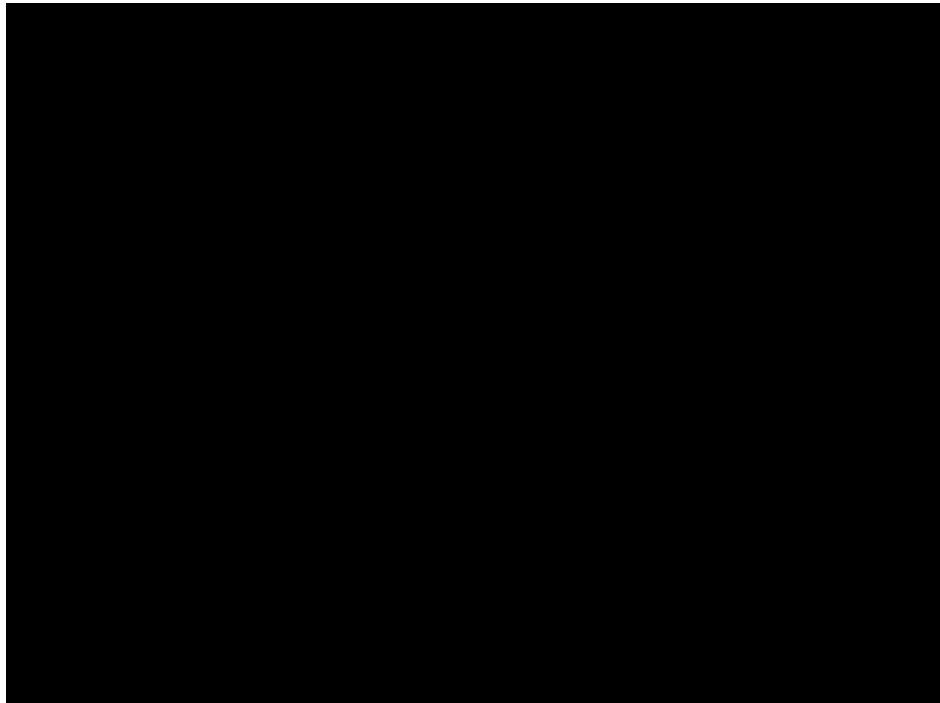
    ShaderUtilities.testShaderProgram(myProgram);
}
```

Mit der Funktion **testShaderProgram** haben wir eine Möglichkeit, Syntax und Ausführbarkeit eines Shaders zu testen.

Gelbes Quad rotieren lassen (Java)

Mit **renderLoop** wollen wir jetzt die Methode implementieren, die für die Visualisierung zuständig ist. In einer Schleife wird dabei das Quad rotiert und durch den Shader visualisiert:

```
public void renderLoop() {  
    prepareShader();  
  
    glEnable(GL_DEPTH_TEST);  
    long start = System.nanoTime();  
    while (!Display.isCloseRequested()) {  
        double now = (System.nanoTime() - start) / 1e9;  
        POGL.setClearColorClearDepth(0.0f, 0.0f, 0.0f);  
  
        glMatrixMode(GL_PROJECTION);  
        glLoadIdentity();  
        double r = (double)getHeight()/getWidth();  
        glFrustum(-1, 1, -r, r, 1, 20);  
        glMatrixMode(GL_MODELVIEW);  
        glLoadIdentity();  
        glTranslated(0, 0, -3);  
        glRotatef((float) now * 5.0f, 0, 0, 1);  
        glRotatef(10 * (float) now * 5.0f, 1, 0, 0);  
  
        POGL.renderViereck();  
        Display.update();  
    }  
}
```

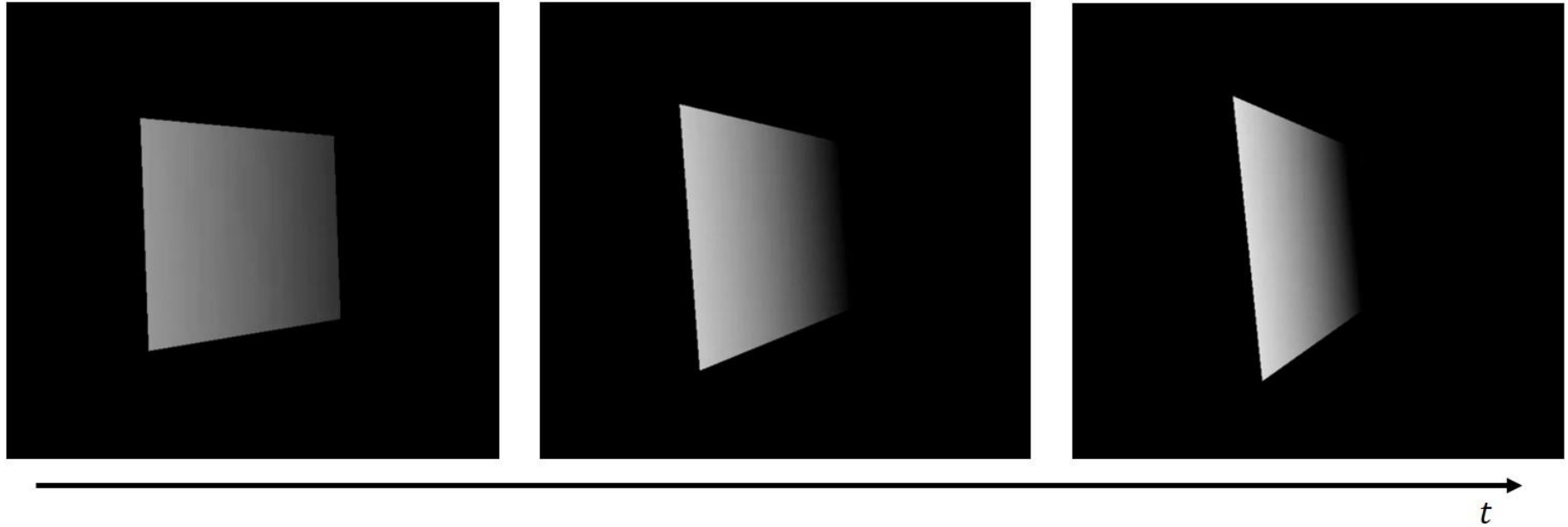




Jetzt wollen wir Farbe und Tiefe kombinieren.

Quad mit Tiefe einfärben

Im folgenden Beispiel wollen wir die Tiefeninformationen, also den **z-Wert** der rotierenden Vertices, nehmen und diesen in einen **Grauwert color** umwandeln. Damit haben wir eine direkte Abbildung einer Rauminformation in einen Farbwert.



Quad mit Tiefe einfärben (Shader)

Die Eckpunkte sollen dabei einen Grauwert erhalten, der mit größerem Abstand abnimmt. Wir drehen uns die z-Werte mit $zVal = 1 - z/2$ zurecht. Der Fragmentshader kennt den Parameter **color** ebenfalls und setzt die Farbe direkt:

```
varying vec4 color;

void main() {
    float zVal = 1.0 - (gl_ModelViewProjectionMatrix * gl_Vertex).z * 0.5;
    color = vec4(zVal, zVal, zVal, 1.0);
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```

Mit dem Schlüsselwort **varying** ([deprecated](#)) können wir also selbst Informationen an Vertices kleben und durch diese direkt durch die Pipeline schleusen:

```
varying vec4 color;

void main() {
    gl_FragColor = color;
}
```



Auch Sprachen, wie OpenGL und GLSL sind
immer in Bewegung ...

Quad mit Tiefe einfärben (Shader)

Die Eckpunkte sollen dabei einen Grauwert erhalten, der mit größerem Abstand abnimmt. Wir drehen uns die z-Werte mit $zVal = 1 - z/2$ zurecht. Der Fragmentshader kennt den Parameter **color** ebenfalls und setzt die Farbe direkt:

```
out vec4 color;

void main() {
    float zVal = 1.0 - (gl_ModelViewProjectionMatrix * gl_Vertex).z * 0.5;
    color = vec4(zVal, zVal, zVal, 1.0);
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```

Zusätzliche Werte werden heute über die Schlüsselwerte **in** und **out** zwischen dem Vertex-Shader und dem Fragment-Shader durch den Rasterisierer geschickt:

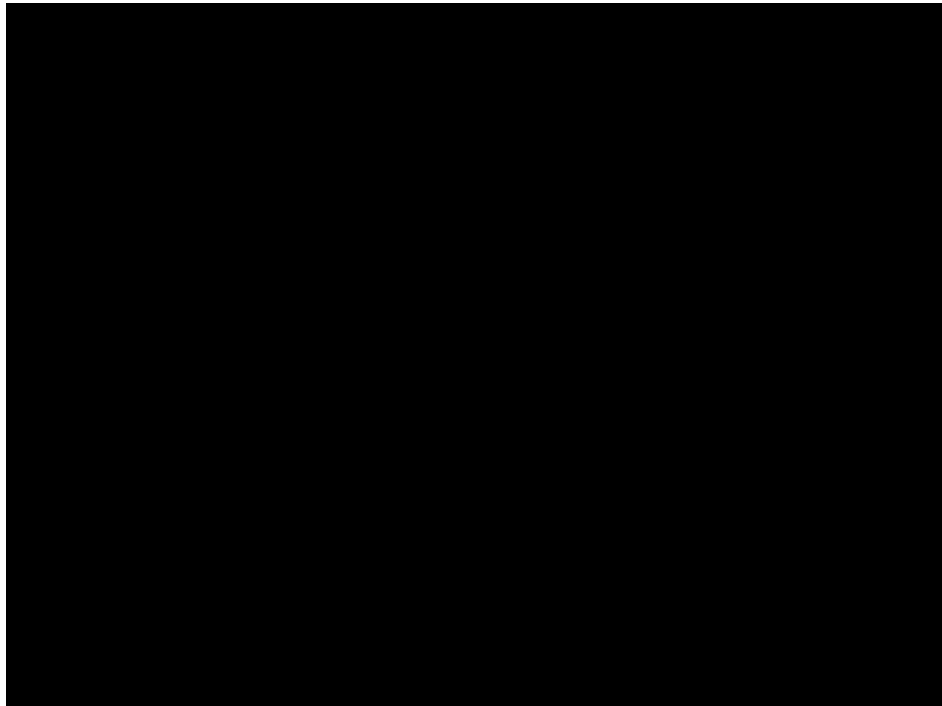
```
in vec4 color;

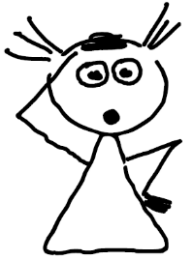
void main() {
    gl_FragColor = color;
}
```

Quad mit Tiefe einfärben (Java)

Mit **renderLoop** wollen wir jetzt die Methode implementieren, die für die Visualisierung zuständig ist. In einer Schleife wird dabei das Quad rotiert und durch den Shader visualisiert:

```
public void renderLoop() {  
    prepareShader();  
  
    glEnable(GL_DEPTH_TEST);  
    long start = System.nanoTime();  
    while (!Display.isCloseRequested()) {  
        double now = (System.nanoTime() - start) / 1e9;  
        POGL.setClearColorClearDepth(0.0f, 0.0f, 0.0f);  
  
        glMatrixMode(GL_PROJECTION);  
        glLoadIdentity();  
        double r = (double)getHeight()/getWidth();  
        glFrustum(-1, 1, -r, r, 1, 20);  
        glMatrixMode(GL_MODELVIEW);  
        glLoadIdentity();  
        glTranslated(0, 0, -3);  
        glRotatef((float) now * 5.0f, 0, 0, 1);  
        glRotatef(10 * (float) now * 5.0f, 1, 0, 0);  
  
        POGL.renderViereck();  
        Display.update();  
    }  
}
```





Konstante Helligkeit spendieren

Quad mit konstanter Helligkeit (Shader)

Wir können jedem Objektpixel über den zusätzlichen Parameter **ambient** eine konstante Helligkeit spendieren:

```
out vec4 color;

float ambient = 0.2;

void main() {
    color      = vec4(1.0, 1.0, 1.0, 1.0) * ambient;
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



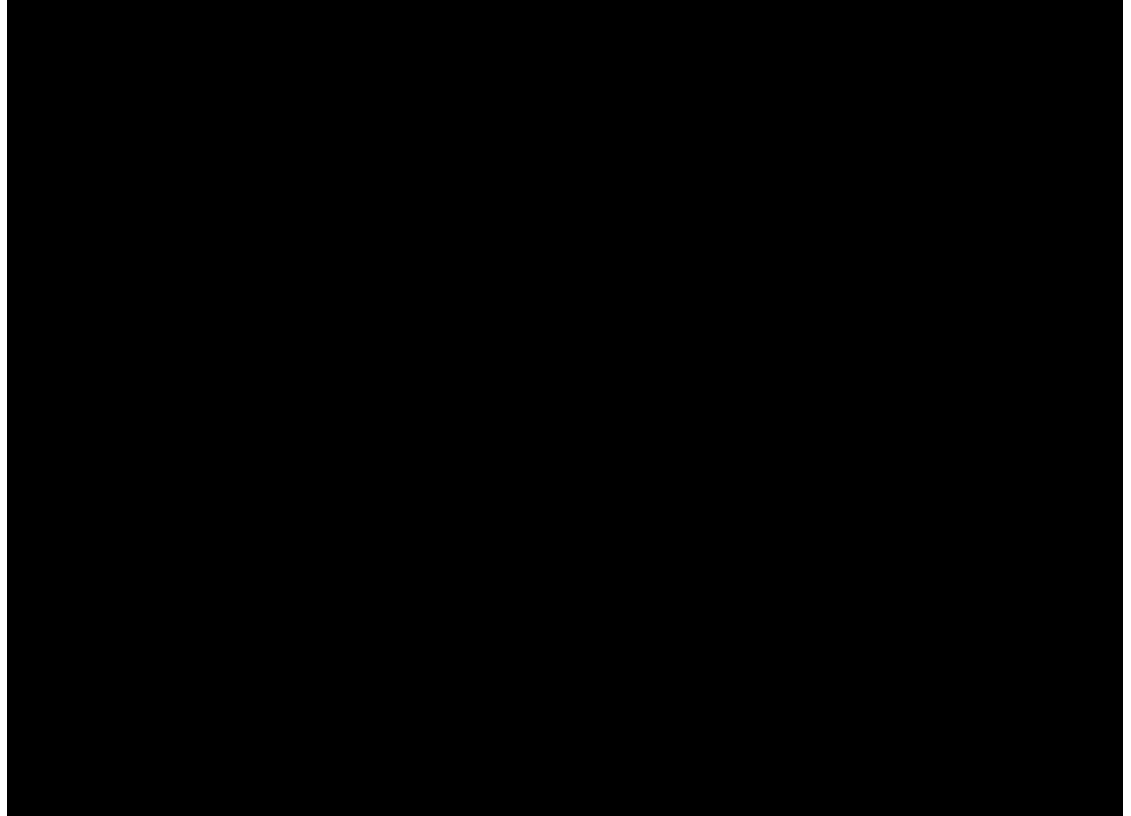
Zusätzliche Werte werden heute über die Schlüsselwerte **in** und **out** zwischen dem Vertex-Shader und dem Fragment-Shader durch den Rasterisierer geschickt:

```
in vec4 color;

void main() {
    gl_FragColor = color;
}
```

Quad mit konstanter Helligkeit (Java)

Schauen wir uns das Ergebnis an:





Licht- und Materialeigenschaften

Quad Licht- und Materialeigenschaften (Shader)

Üblich ist es, Licht- und Materialeigenschaften anzugeben:

```
out vec3 color;

vec3 ambientLight    = vec3(1.0, 1.0, 1.0);
vec3 ambientMaterial = vec3(0.2, 0.2, 0.2);

void main() {
    color          = ambientLight * ambientMaterial;
    gl_Position    = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```



Zusätzliche Werte werden heute über die Schlüsselwerte **in** und **out** zwischen dem Vertex-Shader und dem Fragment-Shader durch den Rasterisierer geschickt:

```
in vec3 color;

void main(void) {
    gl_FragColor = vec4(color, 1.0);
}
```



Eigenschaften LIVE verändern

Torus-Eigenschaften live verändern (Shader)

Wir können über **Uniforms** in Shader-Parameter live eingreifen!

```
out vec4 color;

uniform float ambient;

void main() {
    color = vec4(1.0, 1.0, 1.0, 1.0) * ambient;
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```

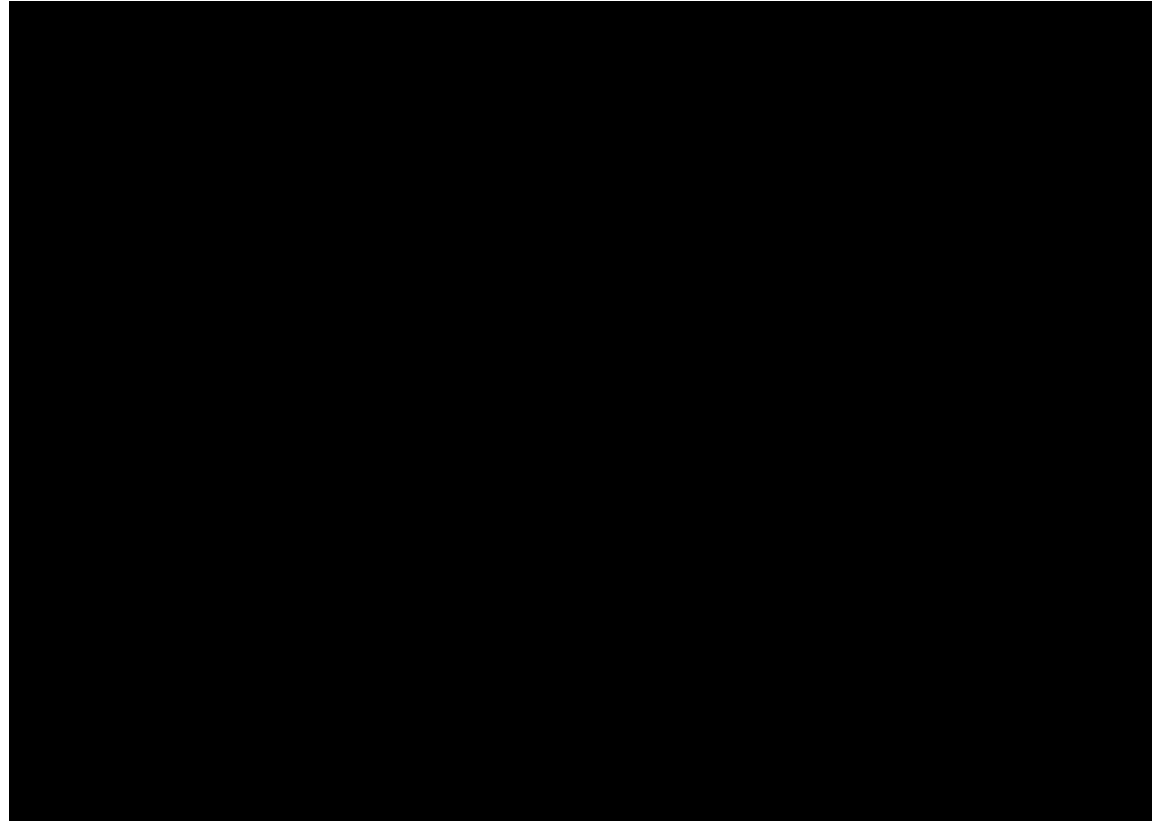
Uniforms erlauben es, Shaderparameter zur Laufzeit direkt aus dem Host-Programm zu verändern. Auf diese Weise können Material- oder Beleuchtungseigenschaften interaktiv angepasst werden, ohne den Shader neu zu kompilieren. Der Fragment-Shader bleibt hier unverändert:

```
in vec4 color;

void main(void) {
    gl_FragColor = color;
}
```

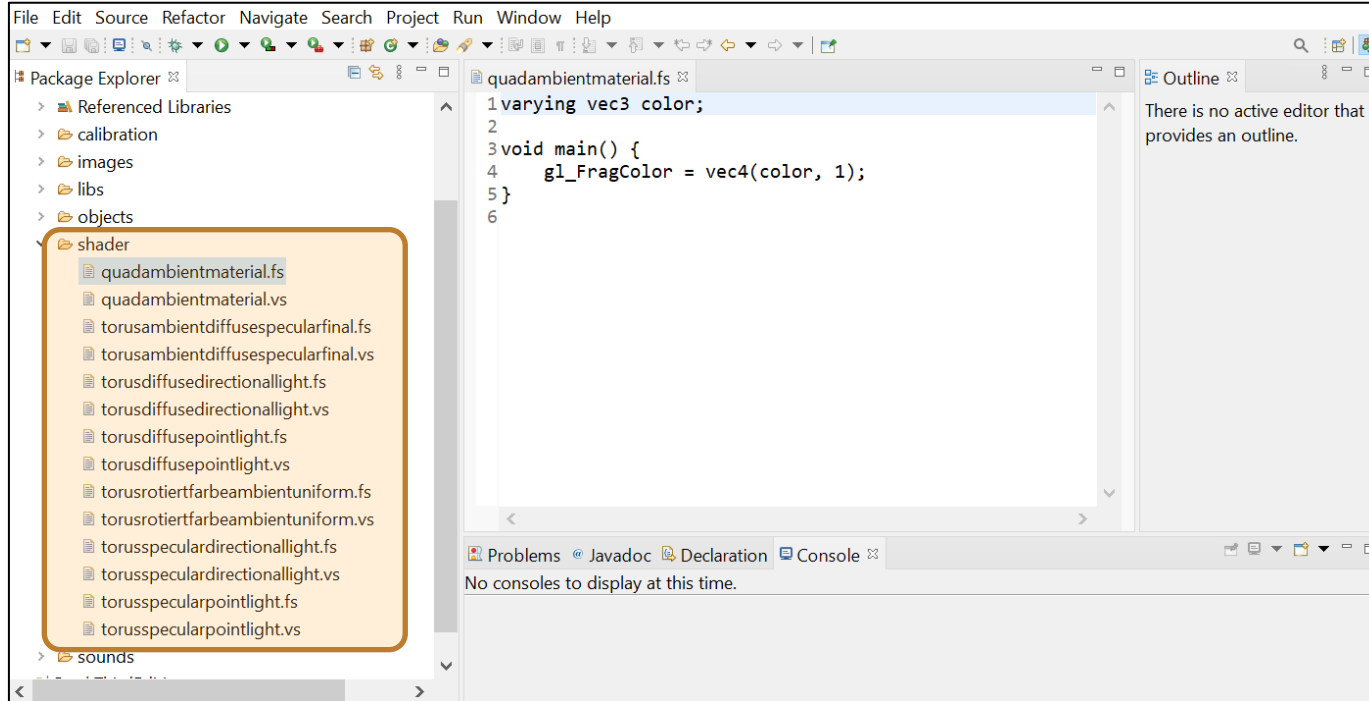
Torus-Eigenschaften live verändern (Java)

Schauen wir uns das Ergebnis an:



Auslagern des Shadercodes

Sinnvoll ist es, den **Shadercode auszulagern** (*.fs für Fragment-Shader und *.vs für Vertex-Shader) und in die Anwendung einzulesen.





Schauen wir uns noch kurz die
[ShaderUtilities](#) an ...

ShaderUtilities – Shadercode einlesen

Wir können den Shadercode sehr einfach zeilenweise als Zeichenkette einlesen:

```
public static String readShadercode(String file) throws Exception {  
    FileReader fi;  
    BufferedReader in;  
    String line, content = "";  
  
    try {  
        fi = new FileReader(file);  
        in = new BufferedReader(fi);  
        while ((line = in.readLine()) != null) {  
            content += line;  
            System.out.println(line);  
        }  
        in.close();  
        return content;  
    }  
    catch (Exception e) { throw new Exception("File not found"); }  
}
```


ShaderUtilities – Shadercode testen

Shaderprogrammierung „blind“ durchzuführen ist sicherlich nicht einfach. Gut, dass wir wenigstens eine Fehlermeldung vor dem eigentlichen Start erhalten können:

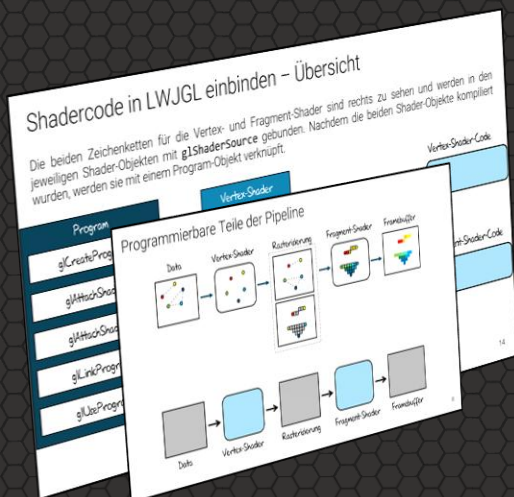
```
public static void testShaderProgram(int shader) {
    errorBuffer.rewind();
    glGetProgram(shader, GL_LINK_STATUS, errorBuffer);
    System.out.println(errorBuffer.get(0)==GL_TRUE?"OK":"ERROR");
    int error = errorBuffer.get(0);
    errorBuffer.put(0,1024);
    glGetProgramInfoLog(shader, errorBuffer, infoBuffer);
    if (error!=GL_TRUE) {
        byte bytes[] = new byte[1024];
        infoBuffer.get(bytes).rewind();
        System.err.println(new String(bytes, 0, errorBuffer.get(0)));
    }
}
```



Den Einsatz der ausgelagerten Shader
sehen wir dann ab Kapitel 5

Einführung in Vertex- und Fragment-Shader

Key Takeaways



- Die Renderpipeline transformiert Geometrie schrittweise zu Bildpunkten
- Vertex-Shader berechnen Positionen von Vertices
- Fragment-Shader bestimmen die Farbe einzelner Fragmente
- Werte können vom Vertex- zum Fragment-Shader übergeben und interpoliert werden
- Die Wirkung eines Shaders ergibt sich aus der Kombination beider Stufen
- Uniforms ermöglichen die Übergabe von Parametern aus dem Host-Programm
- Shaderprogramme laufen massiv parallel auf der GPU
- Shader bilden die Grundlage moderner Echtzeitgrafik

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Welche Aufgabe hat ein Vertex-Shader? Geben Sie zur Erläuterung ein möglichst kurzes Code-Beispiel an.
- (2) Welche Aufgabe hat ein Fragment-Shader? Geben Sie zur Erläuterung ein möglichst kurzes Code-Beispiel an.
- (3) Wie lassen sich zusätzliche Informationen vom Vertex- zum Fragment-Shader übertragen, die dann den Rasterisierungsprozess passieren? Erläutern Sie das an einem sinnvollen Beispiel. Dafür müssen Sie keinen Programmcode angeben, es genügt eine aussagekräftige Skizze.
- (4) Welches Konzept unterstützt Sie dabei, die im Shader verwendeten Parameter direkt zu manipulieren? Geben Sie dafür ein kleines Code-Beispiel an.
- (5) Warum ist die ebenfalls gebräuchliche Bezeichnung Pixel-Shader statt der korrekten Bezeichnung Fragment-Shader irreführend? Erläutern Sie den Sachverhalt.



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

- (1) **Aufgabe 1)** [Java] Ändern Sie den Fragment-Shader aus QuadGelb so, dass das Quad nicht gelb, sondern rot, grün oder blau dargestellt wird.
- (2) **Aufgabe 2)** [Java] Übergeben Sie im Vertex-Shader unterschiedliche Farben an die Eckpunkte. Beobachten Sie die Farbverläufe im Fragment-Shader.
- (3) **Aufgabe 3)** [Java] Steuern Sie die Helligkeit des Objekts über einen Uniform-Parameter. Ändern Sie diesen Wert zur Laufzeit.



Vielen Dank für die Aufmerksamkeit